AI- and MBSE-Based Agile Satellite Design

CHAPTER 5

Decomposition of Digital Satellite Design Indicators

5.1 Analysis of Satellite Functions and Indicators

Within the MBSE framework, satellite functional modeling and indicator modeling provide the essential support for moving from a system concept to a verifiable design. Together, they constitute the core pillars of digital design. They not only support the systematic transformation of requirements into engineering implementation, but also use formalized methods to map complex aerospace mission objectives accurately into a network of decomposable, traceable, and verifiable functional structures and technical indicators. As satellites are iterated rapidly for scientific exploration, constellation networking, and commercial missions, the industry has placed higher demands on the collaborative efficiency, engineering accuracy, and reusability of integrated model-driven functional and indicator modeling.

Compared with conventional document-driven design methods, model-based functional and indicator modeling improves modeling consistency, information transparency, and traceability. A unified modeling language and toolset also enables multidisciplinary teams to conduct collaborative design, balance indicators, and close the requirements loop early in system design. Digital representations of functional structures and indicator parameters make system design activities visualizable, verifiable, and evolvable, significantly shortening iteration cycles, reducing the cost of late-stage changes, and ensuring that satellite systems remain highly reliable and controllable under rapid development schedules [2].

This chapter places satellite function and indicator modeling at the starting point of the digital design system and follows the modeling thread of “requirements to functions, functions to parameters, and parameters to verification.” Unified modeling specifications and multi-view collaboration mechanisms are introduced to

maintain information consistency and traceability among models at different levels. At the same time, model-driven methods tightly associate requirements, functions, and parameters, improving both the systematic nature of the modeling process and its suitability for engineering practice.

For functional modeling, hierarchical decomposition, functional-interaction modeling, and behavioral-dependency analysis progressively transmit and structure functions from the subsystem level down to the equipment level. For indicator modeling, SysML parametric diagrams and constraint models establish a complete hierarchical allocation chain, accurately projecting system-level indicators onto subsystem constraints and equipment parameters.

During satellite design and modeling, functional decomposition and modeling are the first core activities for realizing mission objectives, supporting structural and behavioral design, and enabling accurate indicator allocation. Hierarchical decomposition and model-based expression clearly define the responsibility boundaries of subsystems and equipment. They also establish functional chains, interactions, and dependency logic early in design, providing a solid foundation for subsequent behavioral modeling, interface design, simulation, and verification. The accuracy and completeness of functional modeling directly affect the rationality of system implementation, the reliability of mission execution, and the efficiency and stability of whole-satellite coordination.

5.1.1 Hierarchical Functional Decomposition Based on Mission Requirements

In satellite system modeling, functional decomposition is the first step in converting higher-level mission requirements into concrete engineering implementation paths. A function is the set of capabilities that a system must exhibit during operation and is therefore the principal carrier connecting the “mission concept” with the “physical design.” The resulting model both represents the capability logic by which the system fulfills its mission and provides a structured

basis for subsequent behavioral simulation, performance verification, and indicator allocation.

For satellites that must respond to complex mission commands, strongly couple multiple subsystems, and operate under strict resource constraints, hierarchical functional modeling is not merely a means of improving design completeness and consistency. It is also a core mechanism for ensuring that the system is collaborative, tailorable, and reusable. The MBSE method provides clear logical conventions and language/tool support for functional-decomposition modeling. Within this framework, a function is generally defined as a unit of behavior performed by a system or one of its components to satisfy a requirement; each functional node is an abstract expression of "what is to be done" [3].

Functional decomposition follows the modeling principles of top-down refinement, structure first, clear interfaces, and no redundancy. Its core purpose is to progressively refine top-level system functions into lower-level functional modules that can be allocated, verified, and coordinated. This process requires not only a clear definition of the essential content of each function, but also precise execution boundaries, rational estimates of resource consumption, explicit control paths, and standardized input/output interfaces [4].

In general, satellite functional modeling is divided into three principal levels: system-level functions, subsystem-level functions, and equipment-level functions, as shown in Figure 5-1. System-level functions normally map directly to mission requirements, including orbit-control capability, attitude control, data exchange, energy balance, and communications assurance. Subsystem-level functions express the responsibilities of specialist systems. For example, the attitude and orbit control subsystem performs attitude determination and trajectory planning; the power subsystem manages energy supply, storage, and power protection; and the telemetry, tracking, and command subsystem performs data uplink/downlink and remote-control response. These functions are normally modeled as composable blocks connected by standardized input/output object interfaces [5].

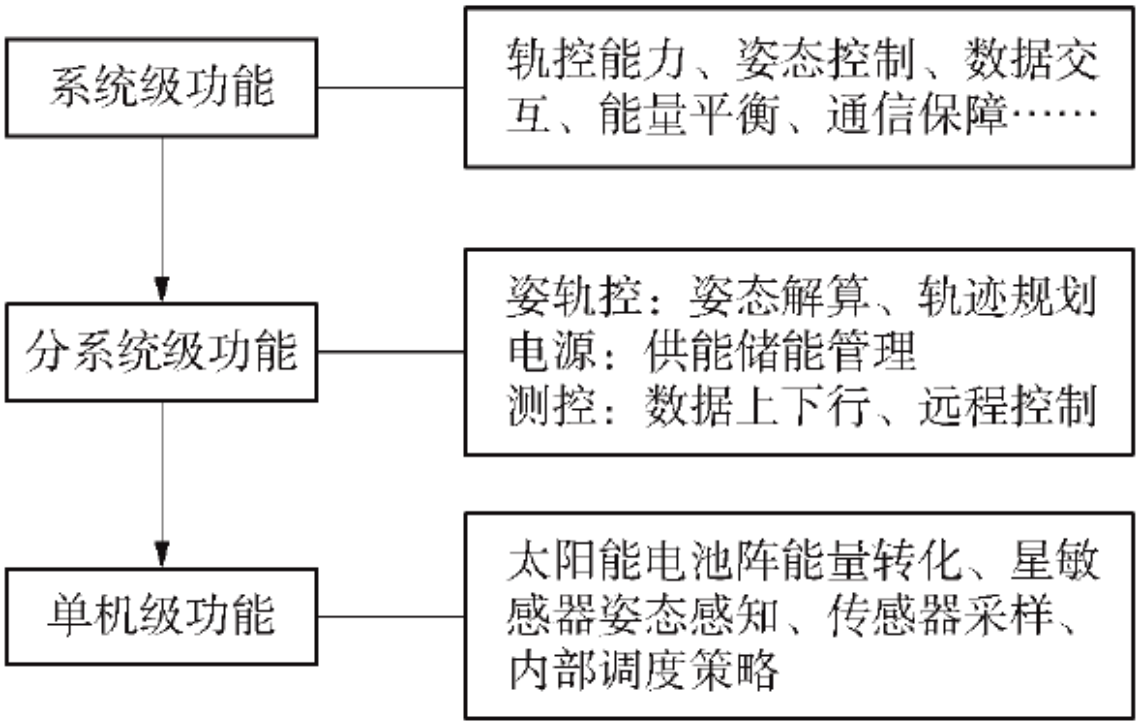

Figure 5-1. Hierarchical decomposition of satellite functions

After further refinement, equipment-level functions describe the behavior of specific devices, such as energy conversion by a solar array, attitude sensing by a star sensor, actuator control, sensor sampling, and internal scheduling strategies. Modeling at this level focuses on execution processes, resource dependencies, and behavioral constraints.

Before constructing a visual functional-decomposition diagram, engineers normally use a structured table to organize the decomposition logic and establish the content baseline. Such a table explicitly defines the name, core responsibility, and boundary conditions of every function at each level and provides an accurate “data baseline” for subsequent graphical modeling. It serves as a blueprint for functional decomposition: by making the inheritance relationships among system, subsystem, and equipment functions explicit, it safeguards completeness and logical consistency. This sequence follows the systems-engineering principle of “logic before form” and gives multidisciplinary teams a shared decomposition baseline, avoiding information distortion caused by ambiguous logic during graphical modeling.

Table 5-1 presents a typical functional decomposition for an attitude-control mission. Hierarchical division makes the logical relationships among levels explicit, helps identify all mission subfunctions, and exposes dependencies among them. Clearly defined input/output relationships connect functions at successive levels into an ordered and complete functional transfer chain. The table progresses from system-level functions to subsystem-level and equipment-level action functions, illustrating how functions are transmitted and implemented during modeling.

功能层级	功能名称	功能描述
系统级	姿态快速调整	在外部任务触发下，完成指定角度范围内的快速姿态机动
分系统级	姿态解算与控制	接收目标姿态、解算轨迹并发出控制命令
分系统级	能源评估与调度	在高功耗任务前确认能源裕度，并调度电池组放电策略
分系统级	指令解析与同步	接收上行指令并与任务时间窗口进行同步
单机级	电机控制环	执行 PID 算法控制飞轮加速曲线，满足姿态机动需求
单机级	模式切换逻辑	在姿控不同工况下切换轮控控制模式
单机级	状态反馈与异常判断	实时采样飞轮状态并判断是否触发保护

Table 5-1. Functional decomposition of an attitude-control mission

Each functional unit in the table can be modeled independently as a functional block and connected to other blocks through interface objects. In practice, engineers must ensure that every functional node has complete input/output boundary conditions, explicit start mechanisms and termination conditions, and a role in subsequent behavioral modeling and simulation verification, including path-coverage checks, abnormal-response design, and resource-balance analysis [6].

Consider the system-level function “global satellite communications coverage and data exchange.” Through a collaborative mission-decomposition mechanism,

this top-level input allocates the macroscopic mission requirement to the communications payload, attitude and orbit control, power, and other subsystems, where it is transformed into subfunctions such as signal modulation/demodulation and energy management. Subsystem-level functions are then refined into execution actions and assigned to equipment such as power amplifiers, star sensors, and solar arrays, defining concrete tasks such as signal-power amplification, attitude sensing and calculation, and conversion of light energy into electrical energy.

Figure 5-2 visualizes the transfer logic of this functional hierarchy. The engineering chain links the system, subsystem, and equipment levels and forms a complete closed loop of functional decomposition, refinement, and implementation. System objectives are progressively mapped to executable engineering behaviors and ultimately assigned to specific equipment functional units. Explicit input/output relationships and interface constraints provide a stable transfer mechanism between levels, ensuring that the transferred information remains consistent and controllable.

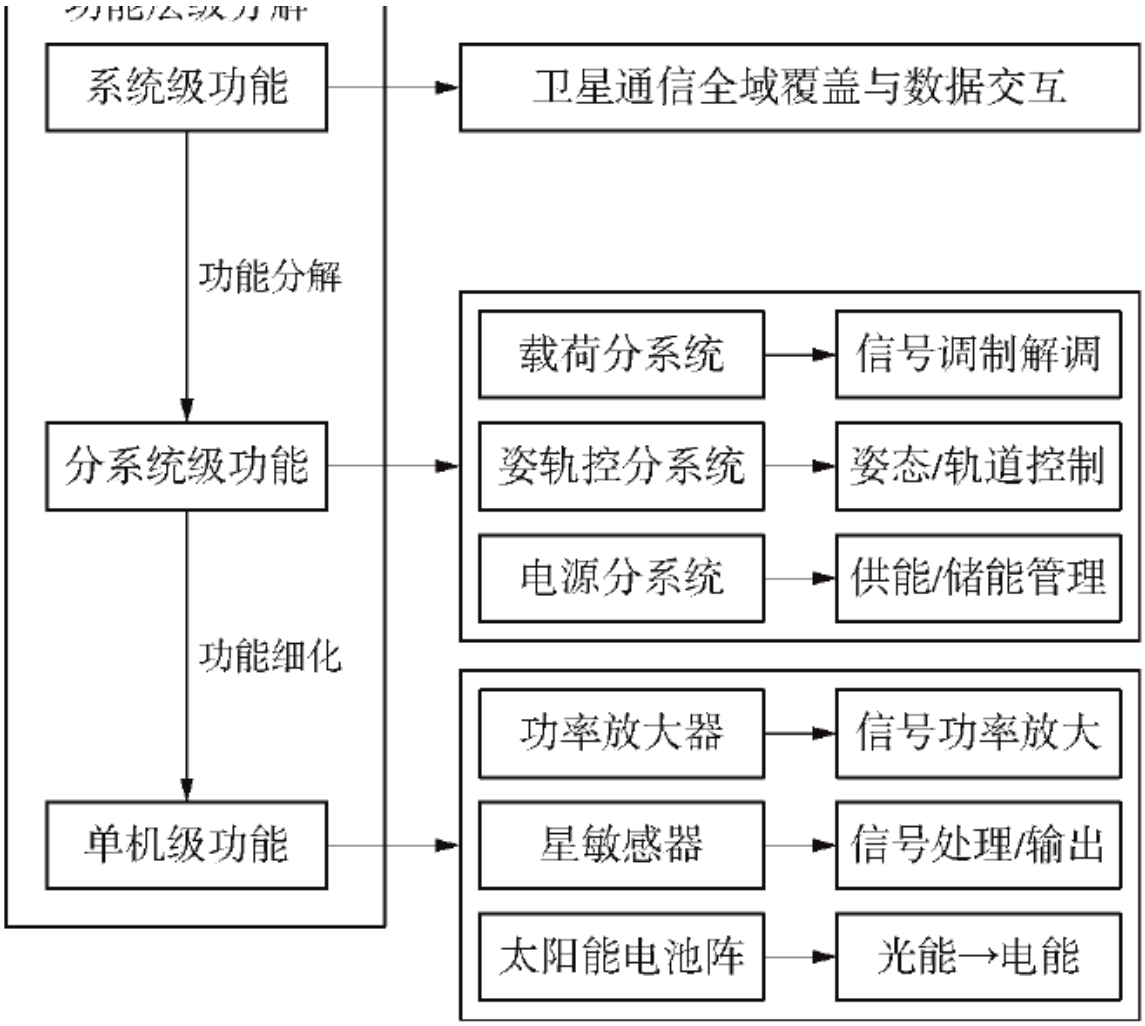

Figure 5-2. Decomposition and transfer logic of the functional hierarchy

In MagicDraw, the Block Definition Diagram (BDD) is one of the core SysML diagram types used to visualize structured system hierarchies. “Blocks” abstractly represent functional units, components, or subsystems, while decomposition relationships clearly show how the system expands from the whole to its constituent parts. A BDD can also combine value-property definitions and interface constraints to model the internal structure more precisely, allowing the model to carry semantic information while expressing structural relationships.

Figure 5-3 visualizes the preceding decomposition logic as a BDD. The block hierarchy and explicit decomposition relationships make it possible to trace rapidly how top-level requirements become specific equipment actions. This gives intuitive

functional-architecture support to subsequent indicator allocation, multidisciplinary collaborative design - for example, coordination of interfaces between the communications payload and the attitude and orbit control subsystem - and model verification, such as simulated confirmation that every module satisfies top-level performance requirements.

通过将上述的功能分解逻辑可视化为图 5－3 的 BDD 图，通过块的层级化与分解关系的明确性，就可快速追溯“顶层需求如何转化为具体设备动作”，为后续指标分配、多学科协同设计（如通信

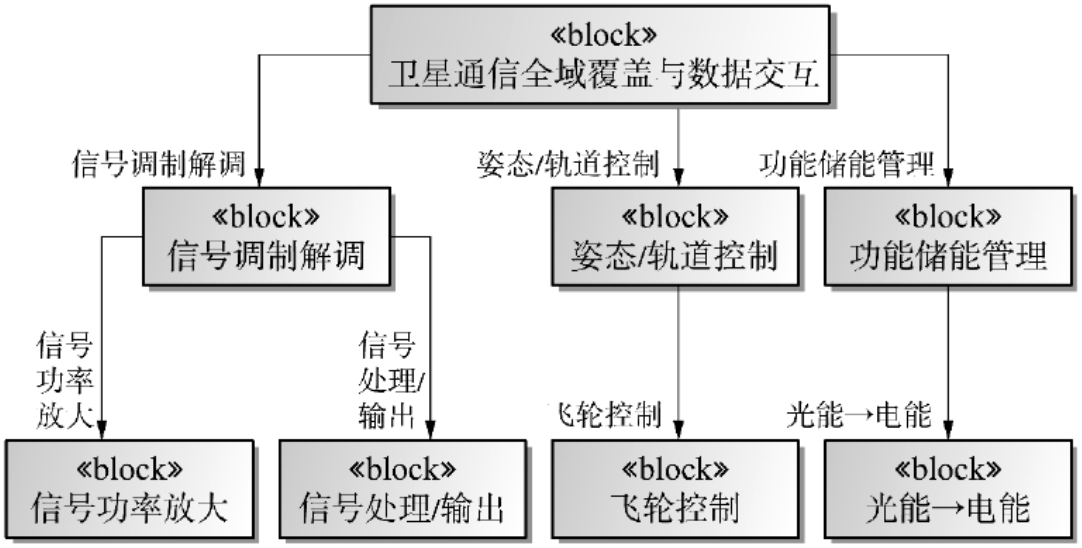

图 5－3　卫星通信功能分解的 BDD

Figure 5-3. BDD for satellite communications functional decomposition

The logical chain of functional decomposition emphasizes clear mapping among functions, behaviors, and resources. A function describes what the system must accomplish; a behavior represents the dynamic process by which that function is realized; and a resource is the physical entity supporting execution of the behavior. Structurally associating these three elements avoids a disconnect between abstract functional descriptions and the implementation layer. The model therefore retains conceptual clarity while remaining implementable in engineering practice. This mapping also provides a common modeling foundation for behavioral modeling,

resource allocation, and performance analysis, improving design consistency and completeness.

Take the “rapid mission-command response” function as an example (Figure 5-4). Its decomposition identifies subfunctions including command parsing, mission configuration, attitude planning, and power-consumption checking. Command parsing converts binary commands uplinked from the ground into executable mission parameters. Mission configuration adjusts the payload operating mode according to the parsing results. Attitude planning computes a trajectory satisfying mission-pointing requirements. Power-consumption checking evaluates whether the energy required during mission execution remains within the power-system margin.

These subfunctions require explicit temporal relationships. For example, power-consumption checking must be performed after attitude planning to confirm that the available energy supports the planned trajectory. Data-interaction paths must also be defined, such as using the output of attitude planning as the input to actuator control. Doing so prevents missing paths and ambiguous dependencies in later behavioral modeling.

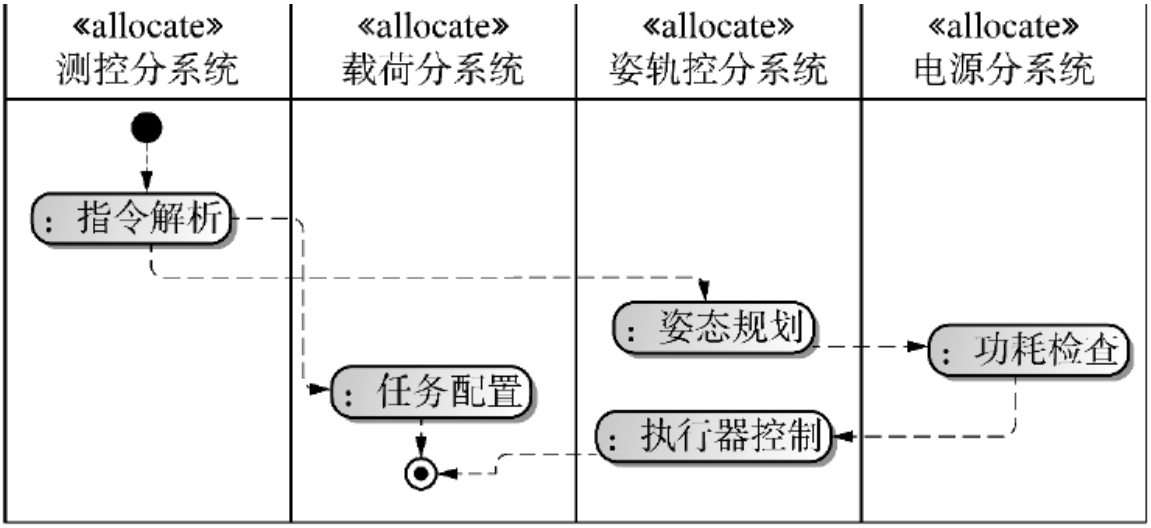

Figure 5-4. Logical chain for rapid response to mission commands

As shown in Figure 5-5, satellites intended for different missions have significantly different functional-decomposition logic, and the model must represent

these differences explicitly. For an Earth-observation satellite, the top-level function “high-resolution imaging coverage” decomposes at subsystem level into high-accuracy pointing by the attitude and orbit control subsystem and functions such as imaging-parameter configuration and exposure control in the payload subsystem. At equipment level, it includes specific actions such as camera shutter control and image-sensor sampling. By contrast, the “inter-satellite data relay” function of a communications relay satellite emphasizes inter-satellite link establishment and data buffering/routing at subsystem level, and focuses on phased-array antenna beamforming and transponder-gain adjustment at equipment level [10].

These mission-oriented differences should be distinguished in the model through attributes of the functional blocks, such as a mission-type tag, so that later design can be adjusted appropriately.

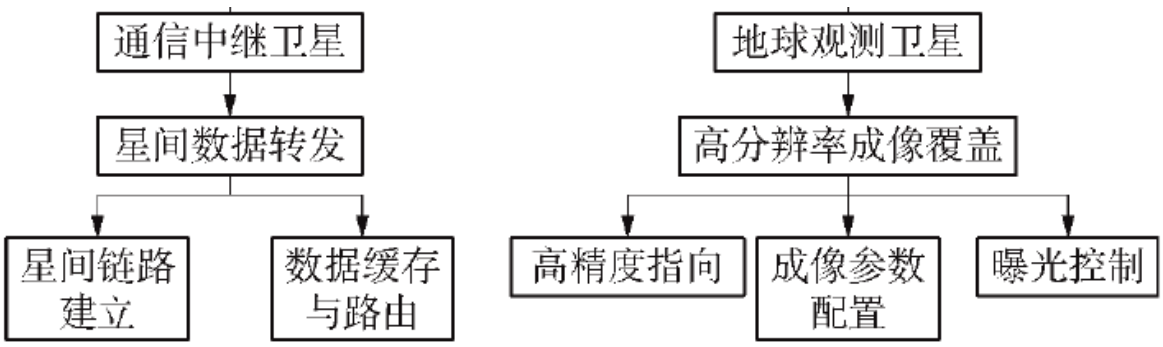

图 5－5　通信中继卫星和地球观测卫星的功能分解对比

在工具支持方面，SysML 中的 BDD 与 IBD 被广泛用于构

Figure 5-5. Comparison of functional decomposition for communications-relay and Earth-observation satellites

In terms of tool support, SysML BDDs and Internal Block Diagrams (IBDs) are widely used to construct functional-decomposition trees and static relationships among functions, while activity diagrams model process-control relationships. Tools based on the ARCADIA methodology, such as Capella, use functional-architecture diagrams and operational scenarios to model functional behavior. These tools emphasize reusability and platform independence of functional units, allowing

models to adapt rapidly to different mission requirements. For example, when building a “power management” function in Capella, a “functional exchange” element can define interaction rules with “payload power supply” and “attitude and orbit control power supply.” When the satellite mission changes from remote sensing to communications, modifying only the functional-exchange thresholds, such as the maximum supply power, can enable rapid module reuse.

Overall, hierarchical functional decomposition is the first stage of system modeling and the key to complete requirements transmission and functional verifiability. A clear system-to-subsystem-to-equipment mapping path establishes a robust and extensible functional framework early in satellite development, laying the foundation for behavioral models, indicator models, simulation, and verification. In practice, the process is iterative. An initial decomposition may omit functions because mission scenarios were not fully understood, in which case collaborative review with the mission-analysis team is needed. Alternatively, an overly coarse functional granularity may prevent allocation to specific equipment, requiring further refinement into subfunctions such as charge control, discharge protection, and power distribution.

5.1.2 Modeling and Analysis of Functional Interactions and Dependencies

Satellite functional modeling must go beyond hierarchical decomposition and structural mapping to capture interactions and dependency mechanisms among functions. Functional-interaction modeling is the key transition from static structure to dynamic behavior. It reveals how multiple functional nodes coordinate in time, resources, and state during mission execution, as well as the constraints governing that coordination.

In highly integrated and strongly coupled satellite systems, subsystem functions often have complex control-flow, data-flow, state-coupling, and resource-contention relationships. When a payload subsystem performs an imaging mission, for example, it must simultaneously invoke high-accuracy pointing by the attitude and orbit

control subsystem, power assurance by the power subsystem, and payload heat rejection by the thermal-control subsystem. The activation sequence, resource occupancy, and status feedback of these functions directly determine mission success. These dynamic relationships must therefore be represented and verified through formal models to support subsequent behavioral simulation, scheduling optimization, and mission reconfiguration.

Within the MBSE framework, functional-interaction modeling emphasizes a behavior-led design concept. Modeling begins with the operational process of the target mission and identifies the dynamic call sequence, input/output logic, execution dependencies, and concurrency conditions among functions. Activity diagrams and sequence diagrams are commonly used. Because an activity diagram can model control flow and data flow simultaneously, it is the most widely used means of representing functional interactions.

An activity diagram uses action nodes and control nodes as its basic building blocks and combines object flows, decision paths, parallel structures, and other modeling elements to describe the conditional logic, inputs and outputs, and process evolution of function calls. An activity is itself a model element with a behavioral process composed of multiple basic actions, which can be described formally through a SysML activity diagram (Figure 5-6). The static hierarchy of activities may be defined in a block definition diagram, while the activity diagram provides a view of the internal behavior of the activity.

The basic elements of an activity diagram include actions, object nodes, object flows, control flows, and control nodes. An action is the fundamental atomic modeling unit of an activity and cannot be decomposed into a smaller modeling unit. Each action must execute to completion without interruption; its execution duration can be ignored at this abstraction level. Every action represents a particular type of processing or transformation. Actions may be further classified as basic actions, call-behavior actions, receive/send-signal actions, and other types, each with a specific modeling symbol.

An object node exists within an activity and most often appears between two actions. It indicates that the first action produces an object token as output and that the next action uses the token as input. An object node is represented by a rectangle. An object flow represents dependencies between activities or states and objects, indicating either that an activity uses an object or that an activity/state affects an object. One object may be manipulated by multiple activities and may appear at multiple points in time as its state changes. The object output by one activity may be the result of the entire activity or may serve as the input to another activity for further use.

A control flow transfers control tokens from one action to the next and therefore describes transitions between actions. In Figure 5-6, control flows are shown as dashed lines with arrows between actions.

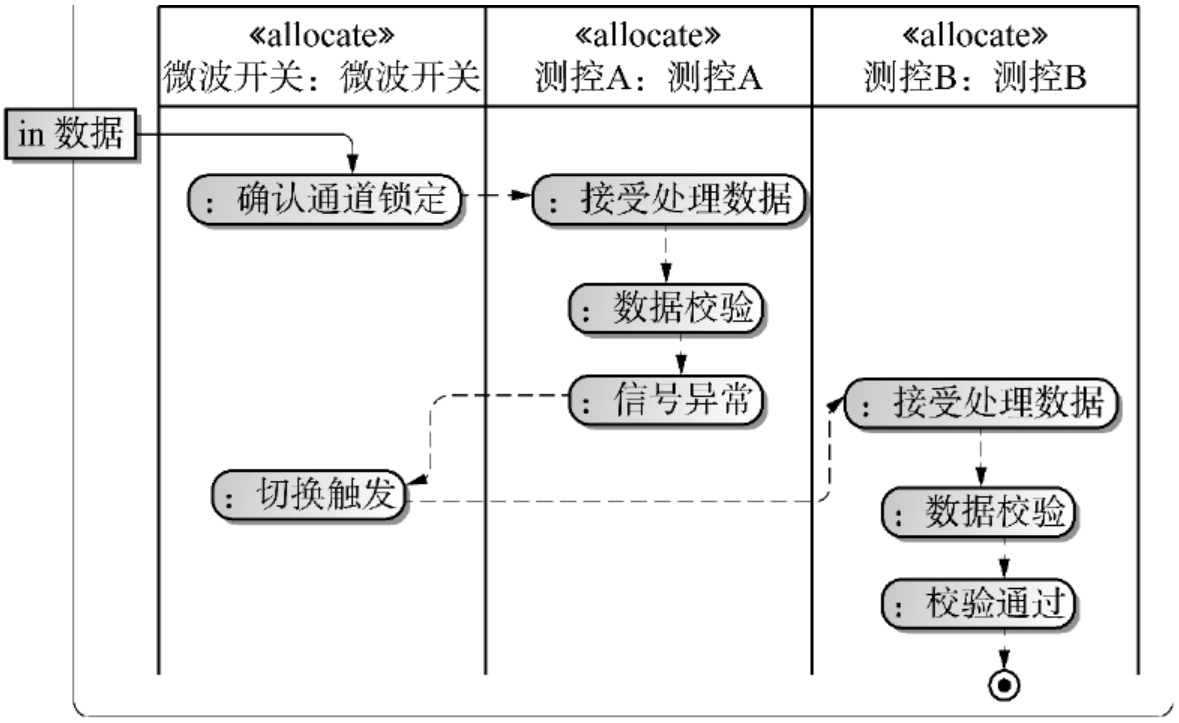

Figure 5-6. Object blocks in an activity diagram

At the tool level, SysML activity diagrams provide a complete modeling chain from functional triggering to execution convergence. Action nodes model functional execution units, control flows express process logic, and object flows define data-

transfer paths. Activity diagrams also support nested modeling, exception-handling models, conditional branches, and other complex behavioral requirements. In complex mission scenarios, multiple activity diagrams can be packaged as a behavior library, reused across system behaviors, and composed into a mission-process system.

MagicDraw's activity-diagram capabilities are well matched to satellite functional-interaction modeling. They focus on internally driven processes and therefore suit core scenarios such as autonomous coordination among satellite subsystems. Their structured nodes map complex interaction scenarios precisely; they support modeling at the system, subsystem, and equipment levels; and their native support for parallel behavior allows clear definition of concurrency conditions and boundaries, helping to prevent resource conflicts. Because the notation draws on conventional flowcharts, it also has a low learning cost and facilitates multidisciplinary communication [15].

Early in modeling, the execution path should first be determined from mission requirements or functional analysis: which functions are serial, which can run in parallel, and where critical data-interaction nodes occur. A top-level activity is then created in MagicDraw as a container and partitioned into swimlanes by system role, so that the behavioral responsibility boundary of each subsystem is explicit.

Each functional action - such as sending a command, collecting data, processing data, or preparing a link - is represented by a behavioral node. Input and output parameters can be defined in the node's properties and bound to actual ports and data types in the structural model. Control flows connect the nodes into a complete behavioral path. Where data must be transferred, object flows and input/output pins bind concrete data variables such as raw images and validation information.

Decision nodes and branch conditions should cover every possible path, including abnormal conditions. Where operations can execute concurrently, fork and

join nodes create the parallel structure; special attention must be paid to synchronization at the end of every concurrent path. MagicDraw provides visual error prompts that help verify closure of the control flow, consistency of inputs and outputs, and the rationality of conditions.

For the data-downlink mission in Figure 5-7, the modeling procedure is: create the top-level activity; define three swimlanes for the onboard computer, integrated electronics, and telemetry/data-transmission subsystem; place behavior nodes in the corresponding swimlanes, such as “send data-acquisition command,” “time synchronization,” and “data packetization”; add control and object flows; introduce a decision point for the “data validation failure” branch; and add a final node to represent completion.

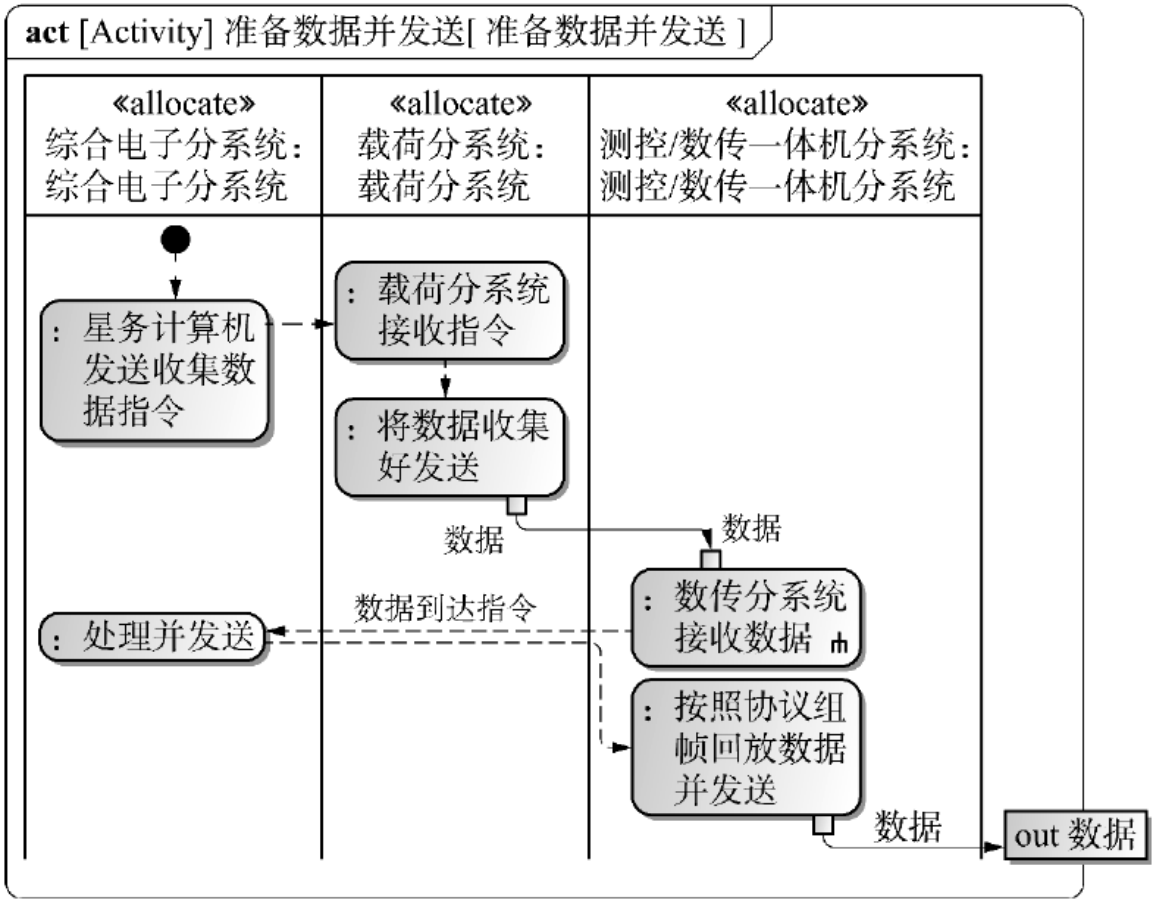

Figure 5-7. Activity for the data-transmission process

After the model is complete, MagicDraw's behavior-verification and simulation functions can traverse the activity paths to confirm that there are no dead paths or dangling nodes and that all critical mission paths are covered. Parameters can also bind the behavioral process to system-resource and performance indicators, enabling further verification of executability and rationality [16].

More complex satellite activities can also be represented. The attitude and orbit control subsystem, for example, must maintain the satellite's pointing and stability from launch-vehicle separation to end of life and is fundamental to stable on-orbit operation. Figure 5-8 shows how an activity diagram makes this complex process intuitive. The temporal logic of subsystem collaboration shows command transmission, state feedback, and fault-tolerance chains among onboard management, attitude and orbit control, power, structure, and other modules during Sun-acquisition mode. It makes clear how multiple systems use a closed loop of sensing, decision, execution, and feedback to support a smooth transition from the safe post-insertion state to efficient operation.

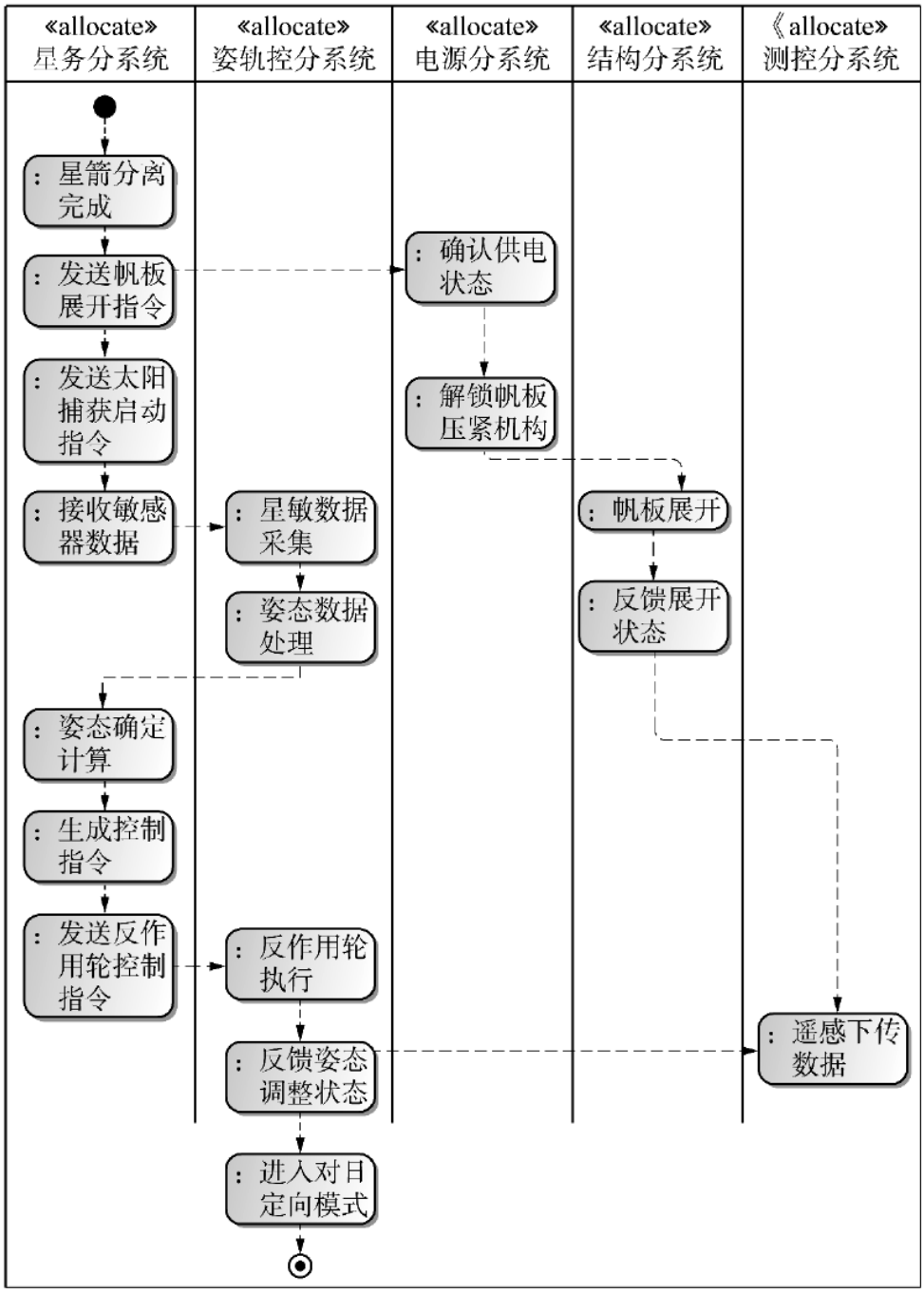

Figure 5-8. Attitude and orbit control workflow activity

The activity model describes not only execution order but also the internal control logic and dependencies through conditional paths, concurrent triggers, and resource checks. This allows system behavior to be fully verified under boundary and abnormal conditions and improves reliability and stability. The model structure can be used downstream to generate behavioral-verification scenarios, extract test

cases, and analyze path coverage. Detailed control paths and resource checks also reveal potential bottlenecks and conflicts, while automated testing tools accelerate verification and improve overall development efficiency.

Modeling functional interactions and dependencies is the principal bridge by which satellite system modeling moves from static structure to dynamic behavior [17]. Formal modeling methods, language mechanisms, and tools allow system engineers to expose the conditional dependencies, behavioral paths, and control relationships underlying functional collaboration. This provides precise support for simulation analysis, scheduling verification, and system integration. The process improves correctness, completeness, and robustness and forms an important part of the digital and closed-loop verification system.

5.2 Satellite Indicator Decomposition and Modeling

During satellite design and verification, decomposition and modeling of performance indicators are essential to requirements closure and design verifiability. A systematic indicator-modeling system progressively transmits the performance constraints, resource boundaries, and functional capabilities specified by upper-level mission requirements and embeds them in the design constraints of system, subsystem, and equipment modules, forming a traceable and verifiable indicator chain. Indicator modeling is not merely a quantitative supplement to functional-structure modeling; it is also a foundational mechanism for coordinating multidisciplinary trade-offs, optimizing resource allocation, and supporting verification analysis [18].

Model-based systems engineering provides formal representation and automated tool support for indicator modeling. Indicator-decomposition diagrams, parametric constraint diagrams, association matrices, and related methods map top-level system indicators accurately to properties of physical components and close the loop with structural parameters and behavioral constraints in the design model. Each indicator node has an explicit source path and scope and can also serve as a

target variable in verification activities, embedded in simulation calculation, performance evaluation, and fault-tolerance analysis.

5.2.1 Hierarchical Indicator Decomposition Based on Functional Analysis

In a digital satellite design system, hierarchical indicator decomposition is the core bridge that transforms top-level mission objectives into executable technical parameters. It follows the principles of function-driven decomposition, hierarchical transmission, and model closure. System modeling languages - particularly SysML parametric diagrams - create a complete chain from macroscopic system indicators to specific equipment parameters. In essence, abstract performance requirements are transformed on the basis of the functional architecture into quantifiable and verifiable design constraints, ultimately forming an indicator network that supports multidisciplinary collaboration and dynamic optimization [19].

Indicator decomposition begins with quantitative definitions of system-level functions. A system-level function is mapped through a SysML parametric diagram into verifiable, measurable technical parameters. Taking attitude-control accuracy as an example (Figure 5-9), the system-level function “high-accuracy inertial pointing” specifies an attitude-pointing error of no more than 1 arcsecond. A constraint equation decomposes this top-level target into subsystem indicators, including attitude-measurement accuracy of no more than 10 arcseconds and attitude-control execution accuracy of no more than 5 arcseconds.

During installation of an optical reference plate, a focal-length accuracy of +/- 1 mm is derived directly from the payload optical-axis pointing requirement. A parametric diagram establishes the mathematical relationship “positioning error = f(attitude error, orbit error),” ensuring closed-loop transmission from functional objectives to engineering parameters. In the modeling tool, such top-level indicators are packaged as constraint blocks and dynamically associated through binding connectors with property parameters in the satellite structural model, forming the first mapping layer for indicator implementation.

联，形成指标落地的第一层映射。

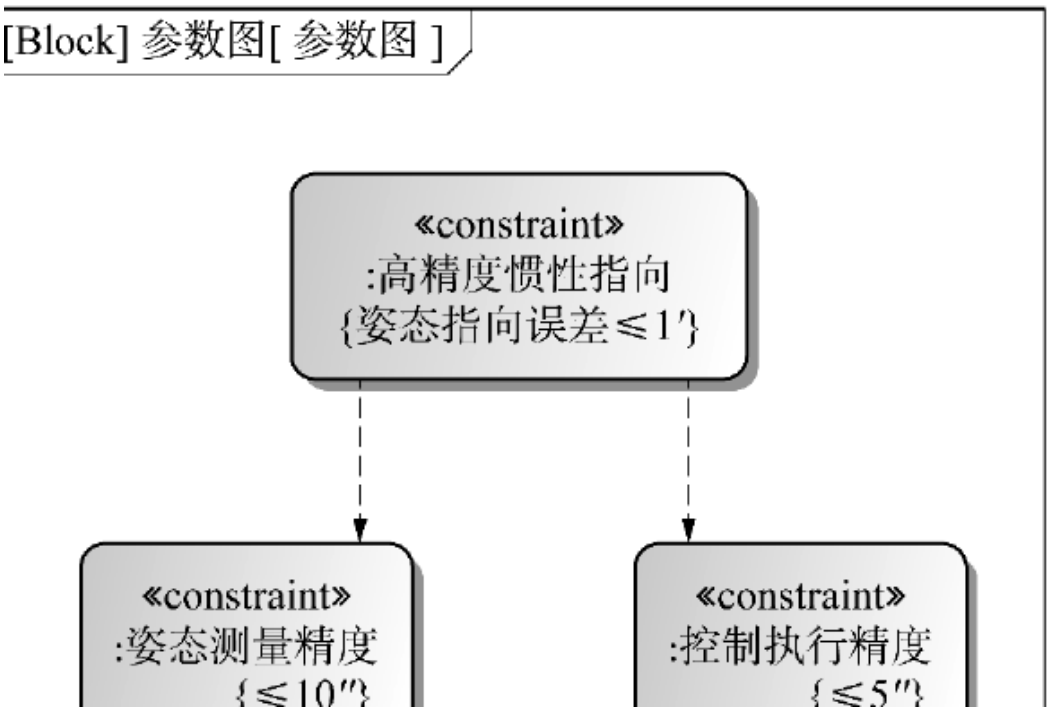

Figure 5-9. Decomposition and association of indicators for acquiring high-resolution surface imagery

Subsystem-level decomposition expresses the transformation logic of specialist domains. When the system-level indicator “positioning accuracy <= 50 m (3 sigma)” is transmitted to the attitude and orbit control subsystem (Figure 5-10), functional analysis and technical feasibility convert it into professional indicators such as attitude-determination accuracy <= 10 arcseconds (3 sigma) and orbit-determination accuracy <= 5 m (3 sigma). This transformation relies on hierarchical transmission in the parametric diagram. The subsystem model establishes a constraint equation of the form “positioning error = f(attitude error, orbit error, time-synchronization error),” using mathematical relationships to determine the contribution of each subsystem indicator.

In the multilevel confirmation system shown in Figure 5-10, a subsystem parametric diagram both inherits system-level indicator requirements and constrains the selection of equipment parameters below it.

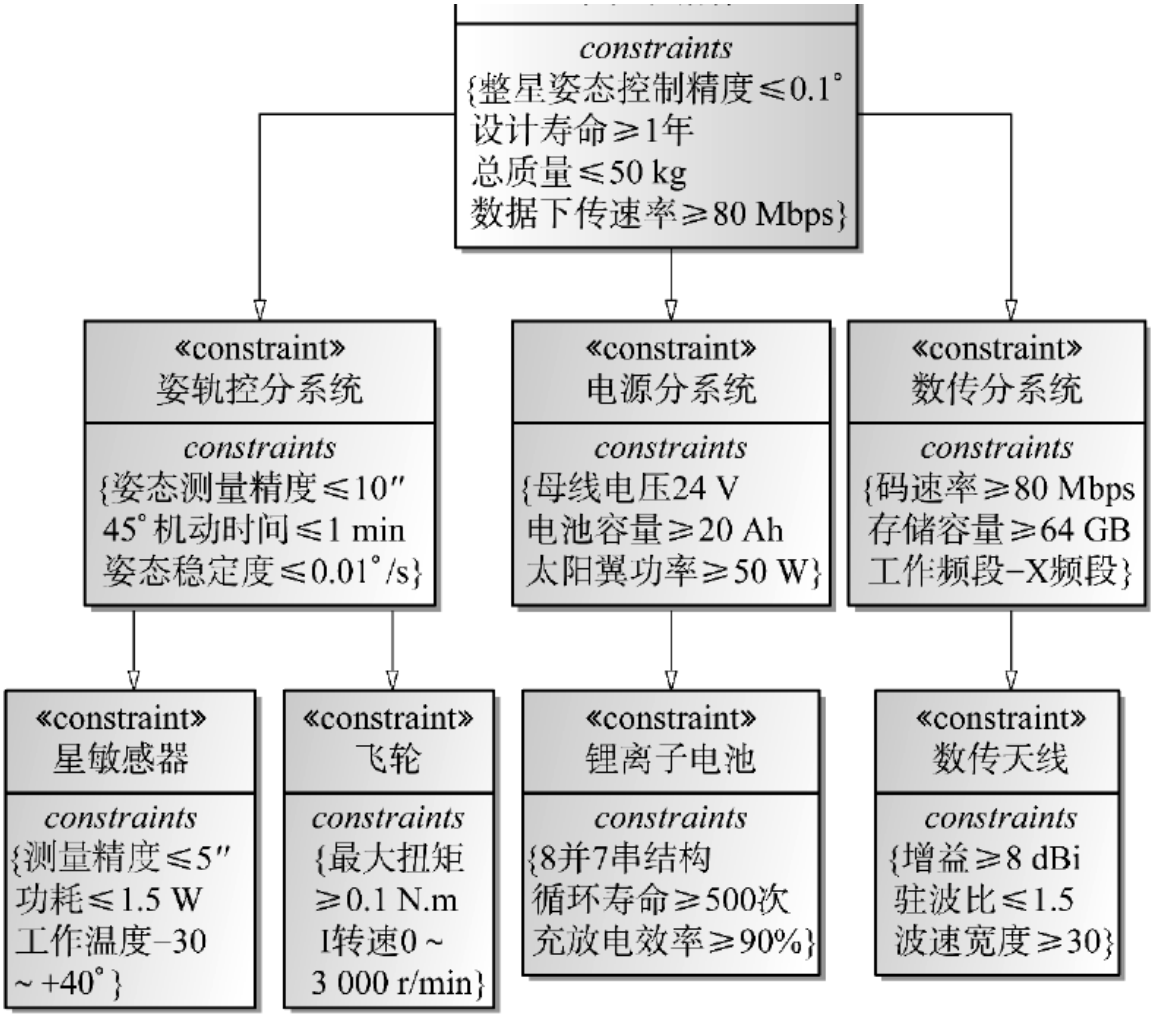

图 5－10　指标分解体系

Figure 5-10. Indicator decomposition system

Equipment-level indicators are the final carriers of implemented design. As shown in Figure 5-11, the subsystem indicator “total mass of the attitude and orbit control subsystem” can be progressively refined through a BDD into executable equipment-level technical parameters, with the quantitative mass target labeled beside each unit. Complex functional units can use secondary submodules to show further details and form a tree hierarchy.

Every equipment indicator is quantified from system-level constraints. If the total mass of the attitude and orbit control subsystem must be less than 6.5 kg, the sum of all equipment mass parameters in the diagram must strictly satisfy this

constraint. Individual targets may include a reaction wheel mass below 0.8 kg and a star-sensor mass below 0.1 kg. Interface data sheets convert these parameters into measurable technical specifications, while SysML constraint blocks perform automatic validation. If an equipment design parameter exceeds its module-level target, the model issues a real-time warning, creating a closed control loop of indicator decomposition, parameter definition, and automatic validation.

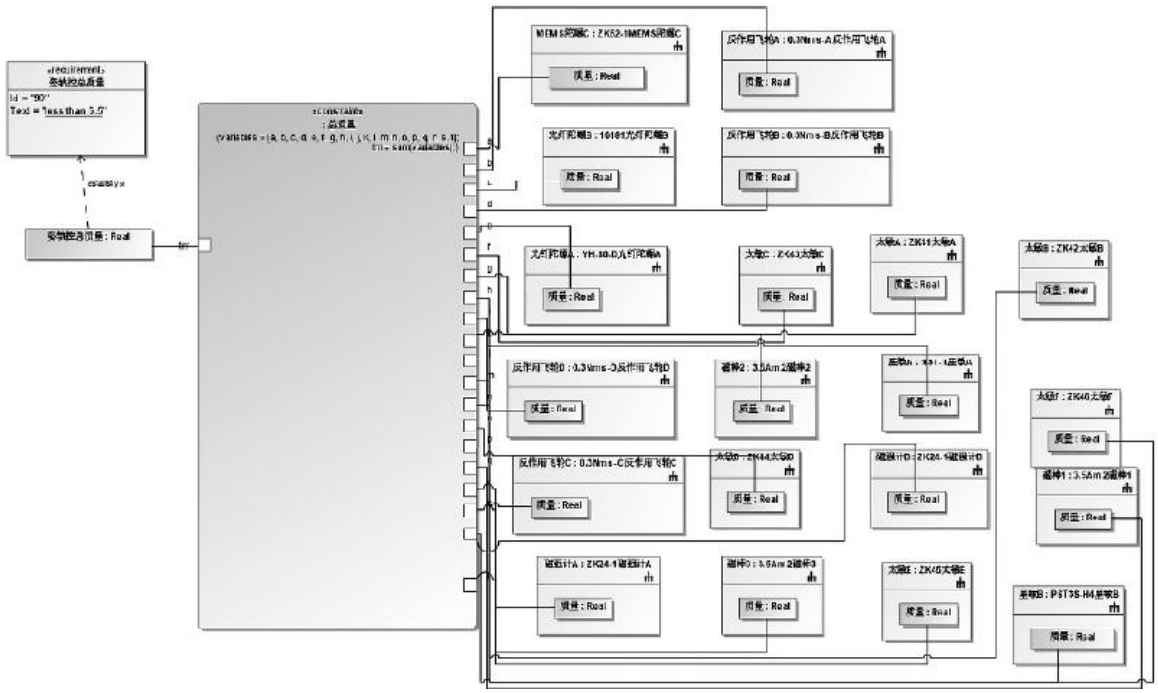

Figure 5-11. Decomposition of the mass indicator for the attitude and orbit control subsystem

System-level indicators vary significantly among satellite types (Figure 5-12) and must be modeled for the particular mission. A scientific-observation satellite may emphasize detection-energy coverage and temporal resolution. An X-ray observation satellite, for example, may specify a detection-energy range of 0.5-8 keV and temporal resolution below 10 microseconds. A commercial communications satellite instead focuses on communications bandwidth and uplink/downlink rate, such as a downlink rate above 100 Mbps and an inter-satellite-link latency below 50 ms.

This distinction should be represented at the top of the indicator model through a mission-type property and should influence the downstream decomposition logic.

A communications satellite's bandwidth indicator is strongly associated with antenna gain and power-amplifier output, whereas a scientific satellite's detection-energy coverage depends on detector material and energy resolution.

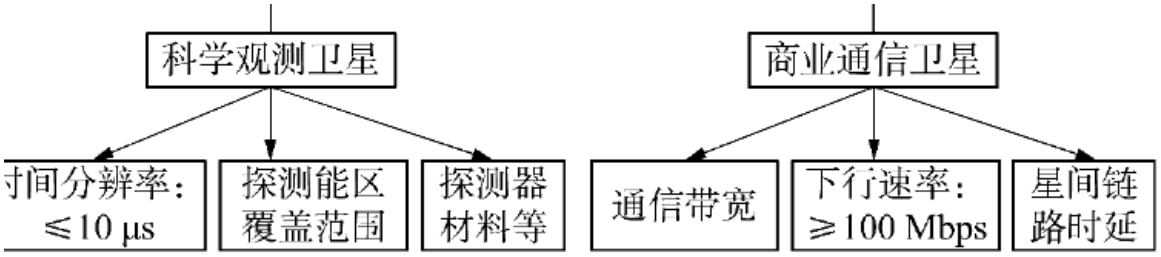

图 5－12　不同类型卫星的指标差异

指标分解的闭环性通过双向追溯实现。正向传递时，系统级
示驱动分系统设计；反向验证时，单机测试数据通过参数图反演

Figure 5-12. Differences in indicators among satellite types

Closed-loop indicator decomposition is achieved through bidirectional traceability. In forward transmission, system-level indicators drive subsystem design. During reverse verification, equipment test data are propagated through the parametric diagram to reconstruct subsystem performance and evaluate system-level mission capability. In the modeling tool, this feedback mechanism appears as real-time solution of the parametric diagram. Whenever an equipment parameter changes, the model automatically recalculates the degree to which upper-level indicators are achieved, avoiding the attenuation of information common in document-based transmission.

For example, when the power subsystem adjusts lithium-battery capacity, the parametric model immediately updates whole-satellite mass and endurance indicators and triggers a multidisciplinary trade-off analysis. Hierarchical indicator decomposition ultimately supports efficient design through three principal properties: traceability across requirements, indicators, and parameters; dynamic response to change; and verifiability.

5.2.2 Modeling and Analysis of Indicator Constraints and Satisfaction

Under the MBSE paradigm, modeling indicator constraints and satisfaction is indispensable to system modeling. Rather than treating indicators as static text or acceptance checklist items, MBSE integrates performance indicators into the model in a formal and structured form, making them engineering entities that persist throughout the system life cycle [20]. Indicators are no longer merely explanatory clauses established during project initiation; in model space they become “active constraints” with explicit locations, mathematical relationships, and verification paths, participating in design, analysis, verification, and evolution.

The core concept of indicator modeling is to transform design requirements into model-controllable and quantifiable constraints. These requirements usually originate in critical user needs or mission indicators and may concern accuracy, power consumption, response time, structural strength, reliability, and other performance requirements or boundary conditions. Under MBSE semantics, they are parsed into quantifiable parameter variables. Constraint blocks then encapsulate their mathematical relationships and rules in a standardized form, and parametric diagrams bind the constraint blocks to specific model parameters, creating a dynamic bridge among structure, behavior, and performance.

This mechanism allows the model to describe not only what the system consists of and what it does, but also how well it performs and whether it satisfies its requirements. A parametric diagram is therefore not merely a tool for indicator modeling; it is the core carrier of indicator tracking and closed-loop verification. Engineers can model complex design objectives visually as constraint expressions and couple them strongly to component or function models. Solvers provided by the modeling tool can calculate and solve the constraint logic in real time, producing immediate feedback on the design.

In essence, a parametric diagram is a special type of internal block diagram used primarily to describe system constraints. Mathematical expressions are encapsulated in constraint blocks, and their variables are represented as constraint

parameters. Instantiating a constraint block binds these parameters to the value properties of system blocks, constraining those blocks and supporting engineering analysis across the system.

A constraint block is a special block that defines reusable constraint expressions. It contains two types of elements: constraint expressions and constraint parameters. A constraint expression may be an equation or inequality governing the mathematical relationships among block value properties. A constraint parameter is a variable used in the expression and receives its value from the bound value property being constrained. A constraint block may contain one or more expressions and associated parameters; all parameters must satisfy the specified equations or inequalities. Importantly, a constraint expression is a declaration, not an assignment. By default, therefore, no dependency is assumed among the parameters and no distinction is made between independent and dependent variables.

Figure 5-13 presents a constraint block for a star sensor. The block constrains the sensor's mass to the interval from 0.07 to 0.09 kg.

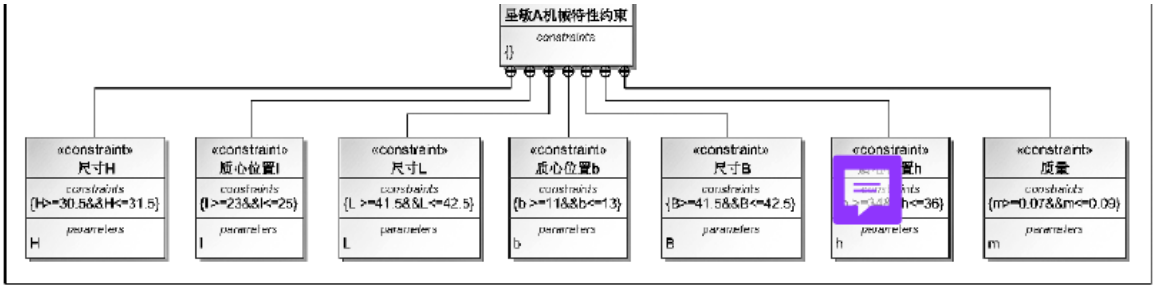

图 5－13　星敏感器约束模块

Figure 5-13. Star-sensor constraint block

Indicator modeling is not performed in isolation. It usually interweaves with requirements, behavioral, structural, and simulation models to form a unified system model. When constraints are embedded in state transitions, behavioral paths, or interface definitions, the parametric diagram can participate directly in system simulation, verification scenarios, and test-case generation. A transition in a state diagram may, for example, be limited by physical indicators such as power

consumption, temperature, or load. Binding the state model to the parametric diagram ensures that its operational logic does not violate established performance boundaries.

Indicator modeling also supports coupled modeling of multiple error sources, multiple dependencies, and multiple constraints in complex systems. In a resource-limited satellite, system performance often depends on collaboration among several subsystems. Modeling must therefore address not only compliance of individual variables but also cross-level and cross-disciplinary indicator synthesis and transmission. A system-level attitude-accuracy indicator, for example, may be affected jointly by sensor accuracy, actuator response, structural thermal deformation, and other physical factors. A multivariable constraint model can synthesize these errors, allocate sub-indicators rationally to specific component parameters, and verify performance attainability early in design.

Specific engineering examples are needed to demonstrate and verify this multivariable constraint-modeling method. A representative equipment item makes it possible to show intuitively how system-level indicators are decomposed, mapped, and synthesized in a constraint model and how coupled parameters affect overall performance. The following star-sensor example therefore illustrates how constraints are constructed in a parametric diagram and applied in engineering practice.

In the star-sensor indicator model shown in Figure 5-14, several constraint blocks describe the body dimensions, center-of-mass position, and mass of the device. The model limits the permissible range of every design variable and uses model solving to determine whether the agreed constraints are satisfied.

:，具体限定了每一个设计变量的取值范围，并通过模型求解验证

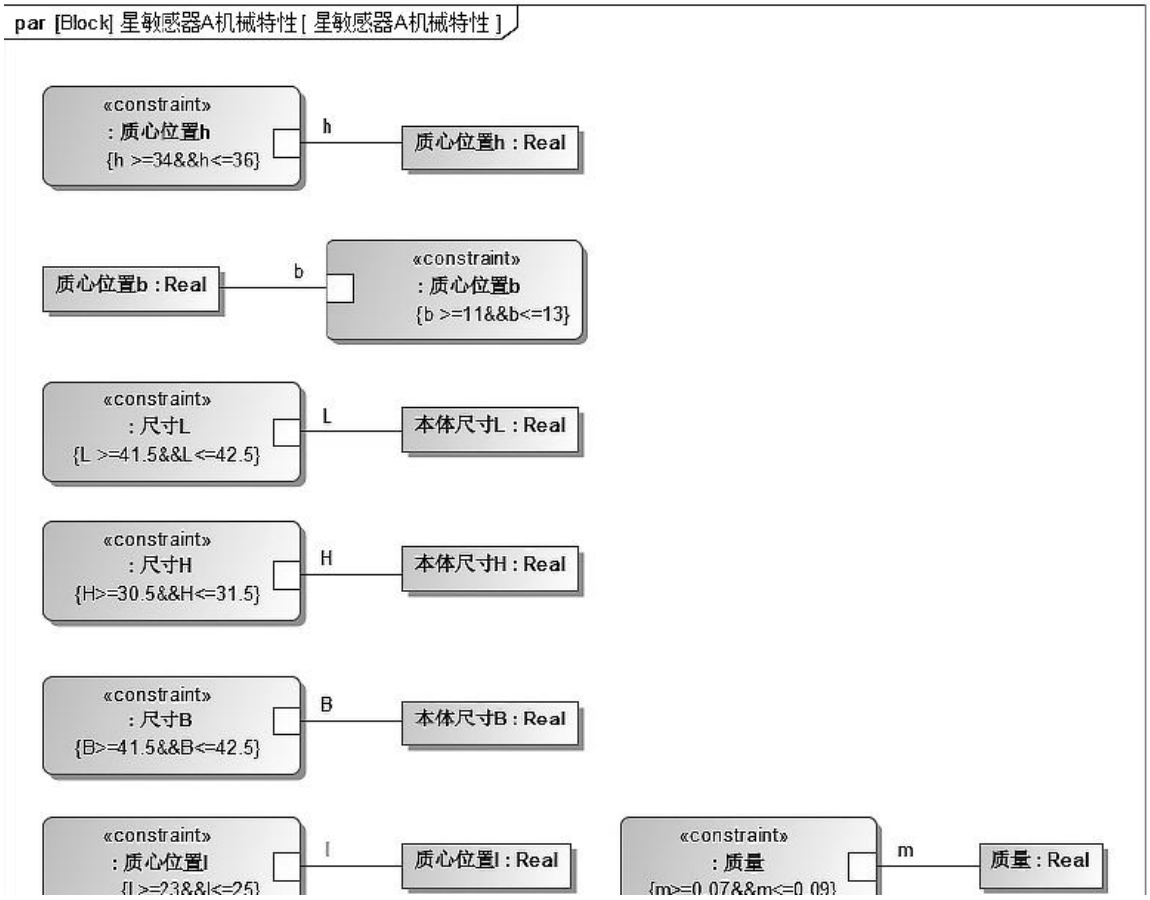

Figure 5-14. Star-sensor parameters

For verification of Star Sensor A, the indicator-confirmation box is displayed in green, showing that its mass property satisfies the performance constraint. This modeling method improves precision and logical clarity and makes every parameter calculable and verifiable. When a parameter changes, the system automatically determines whether the new configuration still satisfies the design constraints, enabling immediate model-level verification, as shown in Figure 5-15.

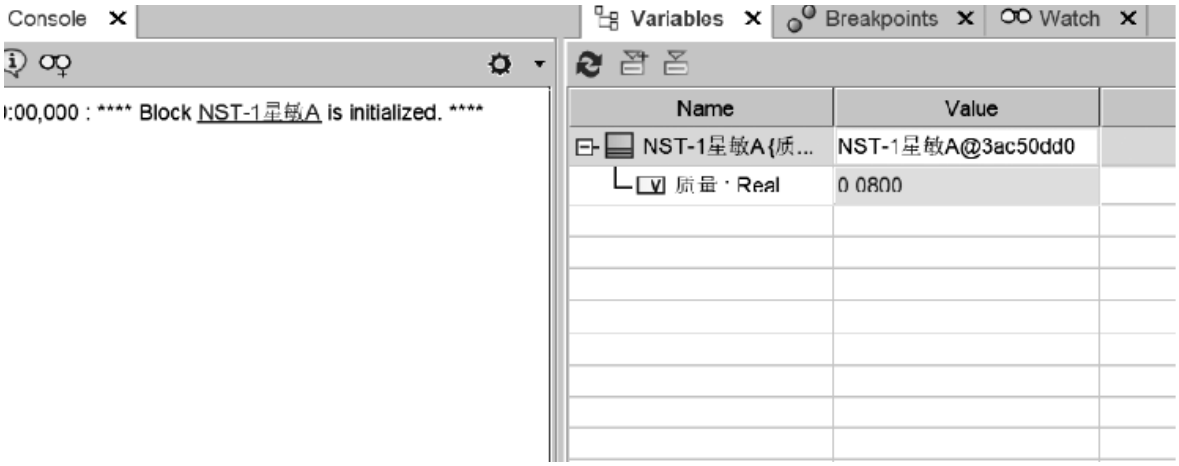

Figure 5-15. Verification of star-sensor indicators

Parametric modeling also supports sensitivity analysis and indicator tuning when design trade-offs and resource conflicts arise. If an indicator exceeds expectations because of material performance or manufacturing error, the model can immediately reveal the impact on total system performance and help determine whether local optimization is insufficient or the overall allocation must be adjusted.

The role of indicator modeling and parametric analysis extends beyond verification to concept evaluation and risk control. Early in system design, different indicator-model structures can be established for competing solutions, allowing rapid comparison of performance boundaries against requirements and supporting quantitative selection. Later in development, the indicator-modeling path provides provenance for parameter changes, shows whether a change violates the original performance target, and supplies a clear basis for quality control and verification attribution.

References

1. Fan Mingjun, Yan Zhen, Li Shihao, et al. Multi-agent collaborative spacecraft-assisted design. Journal of Astronautics, 2025, 46(12): 2495-2504.

2. Zhang Liang, Yao Zhen, Liu Xia, et al. Practice and reflections on digital models in spacecraft systems engineering. Spacecraft Engineering, 2025, 34(06): 55-61.

3. Chen Xiangdong, Miao Yuanming, Fan Haitao. Discussion of issues in the transition of spacecraft development toward MBSE. Spacecraft Engineering, 2025, 34(06): 39-47.

4. Schmid A., Schulte C. Implementing model-based systems engineering for the whole lifecycle of a spacecraft. CEAS Space Journal, 2025.

5. Zhao Lingfeng, Yao Zhen, Fan Haitao. Constructing a collaborative spacecraft MBSE design mode based on enterprise architecture methods. Journal of Astronautics, 2025, 46(09): 1885-1895.

6. Peng Kun, Huang Zhen, He Xiongwen, et al. Research on hierarchical design and verification technology for MBSE-based spacecraft flight plans. Spacecraft Engineering, 2025, 34(02): 8-18.

7. Hu Fang, Shi Guang. Preliminary study of archival management for MBSE model data in product development: A spacecraft-development example. Shandong Archives, 2024(06): 13-17.

8. Wang Yaqiong, Liang Guilin, Zhang Wei, et al. Reflections and recommendations on spacecraft digitalization. Aerospace China, 2024(07): 57-64.

9. Tang Qingyuan, Zhang Yuyuan. A model-based method for generalized spacecraft reliability design and verification analysis. Aerospace Control and Application, 2024, 50(03): 77-85.

10. Hou Xiong, Li Dongfang. MBSE technology and its application in aerospace control. Aerospace Control, 2023, 41(05): 3-13.

11. Peng Kun, Han Dong, Liu Xia, et al. Overall design, simulation, and verification of a crewed lunar-exploration spacecraft based on SysML and Modelica. Journal of Astronautics, 2023, 44(09): 1460-1470.

12. Fan Haitao, Wu Ziwei, Zhang Liang, et al. Spacecraft MBSE application and standardization practice. Aerospace Standardization, 2023(03): 20-23.

13. Wang Wei, Peng Kun. Application of MBSE technology in China's crewed spacecraft development. Spacecraft Engineering, 2022, 31(06): 69-75.

14. Jiang Wei, Luo Jiawei, Wang Zhendong. Research on MBSE-based spacecraft modeling applications. China Plant Engineering, 2021(17): 139-143.

15. Pellerano R. Integrating data-driven and model-based systems engineering for end-to-end small-satellite design. Doctoral dissertation, Politecnico di Torino, Turin, 2025.

16. Yu Guobin. Application and trends of systems-engineering methods for collaboration in deep-space exploration missions. Journal of Deep Space Exploration, 2021, 8(04): 407-415.

17. Jiao Hongchen, Lei Yong, Zhang Hongyu, et al. Exploration of MBSE-based spacecraft system modeling, analysis, design, and development methods. Systems Engineering and Electronics, 2021, 43(09): 2516-2525.

18. Kang Xiaoli. Analysis of the impact of model-based systems engineering (MBSE) on aerospace manufacturing. The Journal of New Industrialization, 2020, 10(08): 93-95.

19. Fan Haitao, Liu Xia, Zhao Lingfeng, et al. Applying MBSE theory and methods to innovative spacecraft research and development. Civil-Military Integration on Cyberspace, 2020(07): 22-26.

20. Zhang Bonan, Qi Faren, Xing Tao, et al. Research and practice on model-based development methods for crewed spacecraft. Acta Aeronautica et Astronautica Sinica, 2020, 41(07): 78-86.